

Purdue Math Club

Problem of the Week

Beginner's Codebase Guide

How the React frontend, Express backend, PostgreSQL database, authentication, grading, and scheduling fit together

This guide assumes you know a little programming, but do not yet know React, Node.js, Express, middleware, JSON Web Tokens, or relational databases.

Generated from the codebase on August 31, 2026

Contents

How to use this guide	iv
1 The system in one picture	1
1.1 The three running pieces	1
1.2 A request from click to database and back	1
2 Repository map	3
2.1 Top-level structure	3
2.2 Important frontend files	3
2.3 Important backend and database files	4
3 JavaScript, TypeScript, Node, and React	5
3.1 JavaScript versus TypeScript	5
3.2 What Node.js is	5
3.3 Asynchronous code: promises, async, and await	5
3.4 What React is	6
3.4.1 Props	6
3.4.2 State	6
3.4.3 Effects	7
4 Frontend architecture	8
4.1 Starting React	8
4.2 Routing and global login state	8
4.3 The shared API helper	9
4.4 Development proxy	9
4.5 Pages and reusable components	10
5 Backend architecture and middleware	11
5.1 What Express adds to Node	11
5.2 Middleware without the jargon	11
5.3 Common middleware in this project	12
5.4 The final error handler	12
6 PostgreSQL and how data is stored	13
6.1 Tables, rows, columns, and keys	13
6.2 The connection pool	13
6.3 Why SQL parameters matter	13
6.4 Data dictionary	14

6.4.1	users	14
6.4.2	problems	14
6.4.3	submissions	15
6.4.4	seasons	15
6.4.5	problem_proposals	16
6.4.6	hint_requests	16
6.5	Joins	16
6.6	Schema versus migrations	16
7	Authentication, verification, and permissions	17
7.1	Password storage	17
7.2	Email verification flow	17
7.3	Password reset flow	17
7.4	JWT login sessions	18
7.5	Why authentication reloads the user	18
7.6	Authentication versus authorization	18
7.7	Rate limiting and DDoS reality	19
8	Feature workflows	20
8.1	Problem scheduling	20
8.2	Computational and proof submissions	20
8.3	Editing a response	21
8.4	Grading	21
8.5	Leaderboard seasons	21
8.6	Hints and proposals	21
9	API reference	22
10	Configuration, running, and testing	24
10.1	Environment variables	24
10.2	Development commands	24
10.3	Database setup	25
10.4	A practical manual test checklist	25
11	How to trace and change the code	26
11.1	Trace a feature vertically	26
11.2	Example: adding a new problem field	26
11.3	Reading an error	27
12	Known tradeoffs and production notes	28
12.1	File storage	28
12.2	Token storage in localStorage	28
12.3	One large backend file	28
12.4	Validation	28
12.5	Time handling	28
12.6	Operational protection	29
A	Glossary	30
B	Fast file lookup	32

Final mental model**33**

How to use this guide

Read Chapters 1–4 in order if web development is new to you. After that, jump to the feature you need. Code excerpts are shortened to emphasize the idea; the filename above each excerpt tells you where to find the complete implementation.

Three words appear constantly:

Frontend Code that runs in the visitor’s browser and draws the interface. In this project, that is React and CSS inside `src/`.

Backend Code that runs on the server, receives HTTP requests, enforces rules, and talks to the database. In this project, that is Node.js and Express inside `server/`.

Database Durable structured storage. PostgreSQL stores users, problems, submissions, seasons, proposals, and hint requests.

Never put database passwords, SMTP passwords, or `JWT_SECRET` values into frontend code. Browser code is downloaded by every visitor. Secrets belong in the server’s uncommitted `.env` file or a production secret manager.

The system in one picture

1.1 The three running pieces

The app is not one program. During development, three pieces cooperate:

Piece	Job	Development address
React + Vite	Draw pages, collect form input, call the API, and remember the logged-in user in the browser.	<code>http://localhost:5173</code>
Express API	Validate requests, authenticate users, apply permissions, run business logic, and return JSON.	<code>http://localhost:3000</code>
PostgreSQL	Persist records after the browser and server stop.	Database named <code>potw</code>

Think of React as the receptionist, Express as the office staff, and PostgreSQL as the filing room. The receptionist never walks into the filing room. It gives a request to the office staff; the staff checks permissions, reads or updates a file, and returns an answer.

1.2 A request from click to database and back

When a solver presses “Submit solution,” this happens:

1. React’s submit handler prevents the browser’s normal page reload.
2. React builds a `FormData` object containing the problem ID, answer, typed work, and optional file.
3. `src/lib/api.ts` sends an HTTP `POST` request to `/api/submissions`. It adds the user’s token to the `Authorization` header.
4. Vite proxies `/api` to port 3000 during development.
5. Express runs middleware in order: request parsing, rate limiting, authentication, file parsing, then the route handler.

6. The route validates the problem and response, then issues a parameterized SQL `INSERT` through the PostgreSQL connection pool.
7. PostgreSQL saves the row and returns its generated ID and timestamp.
8. Express sends JSON such as `{"message": "Your answer was submitted."}`.
9. React updates its state, causing the interface to render the success message.

That pattern—event, request, middleware, SQL, JSON, state update—is the central loop of the entire application.

Repository map

2.1 Top-level structure

```

potw/
|-- db/                PostgreSQL schema and migrations
|-- docs/              This guide (.tex and .pdf)
|-- public/            Files copied directly into the frontend build
|-- server/            Node/Express backend
|   |-- auth.js        JWT creation and permission middleware
|   |-- db.js          PostgreSQL connection pool
|   |-- email.js       SMTP verification/reset messages
|   |-- index.js       API setup and all route handlers
|-- src/               React frontend
|   |-- components/    Reusable UI pieces
|   |-- lib/           Shared API/auth request helpers
|   |-- pages/         One main component per screen
|   |-- types/         Shared TypeScript data shapes
|   |-- App.tsx        Browser routes and global user state
|   |-- main.tsx       Frontend entry point
|-- .env               Local secrets; ignored by Git
|-- .env.example        Safe configuration template
|-- package.json        Dependencies and commands
|-- vite.config.ts      Frontend build and development proxy
|-- README.md          Setup quick reference

```

2.2 Important frontend files

File	Responsibility
src/main.tsx	Finds the HTML element named root and mounts the React application.
src/App.tsx	Defines URL-to-page mappings and owns the logged-in user state.
src/lib/api.ts	Central wrapper around browser fetch; adds headers and turns API errors into JavaScript errors.

File	Responsibility
src/lib/auth.ts	Small functions for login, signup, verification, and password reset.
src/components/problem-widget/CurrentProblems.tsx	Loads current problems, renders MathJax, and submits answers/files.
src/pages/RequestHint.tsx	Separate hint-request form with selectable problem buttons and a LaTeX preview.
src/pages/ViewProfile.tsx	Shows private submission history, scores, feedback, response editing, and anonymous-mode preference.
src/pages/Admin.tsx	Admin tabs for grading, problem scheduling, proposals, seasons, and user roles.
src/pages/Leaderboard.tsx	Loads seasons and spreadsheet-style scores; refreshes every 15 seconds.
src/pages/auth/*.tsx	Signup, login, verification, forgot-password, and reset-password screens.
src/**/*.css	The cream, tan, and dark visual style. CSS changes appearance; it does not store data.

2.3 Important backend and database files

File	Responsibility
server/index.js	Creates Express, installs middleware, declares every API route, validates input, and runs SQL.
server/auth.js	Signs JWTs and defines requireAuth, requireAdmin, and requireSuperuser.
server/db.js	Creates one reusable PostgreSQL connection pool.
server/email.js	Configures Nodemailer once and sends verification/reset links through SMTP.
db/001_schema.sql	Complete schema for a new database.
db/002_sample_problems.sql	Optional starter problem records.
db/003--008_*.sql	Incremental upgrades for an existing database.

JavaScript, TypeScript, Node, and React

3.1 JavaScript versus TypeScript

JavaScript is the language that browsers and Node.js execute. TypeScript adds compile-time descriptions of values. Files ending in `.tsx` may contain TypeScript plus React's HTML-like JSX syntax. The browser never receives TypeScript directly; Vite compiles it into JavaScript.

src/types/user.ts — a TypeScript type

```
1 export type User = {  
2   id: number  
3   username: string  
4   email: string  
5   is_admin?: boolean  
6   is_superuser?: boolean  
7   leaderboard_hidden?: boolean  
8   token: string  
9 }
```

This does not create a database table or runtime object. It tells the TypeScript checker, “whenever we call something a `User`, it should have these fields.” The question mark means a field may be absent.

3.2 What Node.js is

Node.js is a program that executes JavaScript outside a web browser. That makes JavaScript usable for a server. It provides access to networking, environment variables, files, and installed packages. The line below starts the backend with Node:

```
node server/index.js
```

Because `package.json` contains `"type": "module"`, the backend uses modern `import/export` syntax.

3.3 Asynchronous code: promises, async, and await

Database queries and network requests take time. JavaScript does not freeze the whole server while waiting. These operations return a *Promise*, an object representing a future result. Marking a function `async` lets it use `await`:

```
1 app.get('/api/health', async (_req, res, next) => {
2   try {
3     await query('SELECT 1')
4     res.json({ status: 'ok' })
5   } catch (error) {
6     next(error)
7   }
8 })
```

`await` pauses only this request handler until PostgreSQL responds. The server can continue handling other requests. If the query fails, `catch` passes the error to Express's final error middleware.

3.4 What React is

React lets the interface be described as functions of state. A React *component* is usually a function that returns JSX:

```
1 function FrontPage({ user, setUser }: Props) {
2   return (
3     <div className="front-page">
4       <NavBar user={user} setUser={setUser} />
5       <h1>This week's problems!</h1>
6       <CurrentProb user={user} />
7     </div>
8   )
9 }
```

JSX resembles HTML, but it is JavaScript syntax transformed during the build. Braces embed JavaScript expressions. Component names begin with capital letters; lowercase names such as `div` represent browser elements.

3.4.1 Props

Props are inputs passed from a parent component to a child. Above, `FrontPage` gives the current user to `CurrentProb`. Props flow downward. A child should not directly mutate a prop.

3.4.2 State

State is data React remembers between renders:

```
1 const [answerText, setAnswerText] = useState('')
2
3 <input
4   value={answerText}
5   onChange={(event) => setAnswerText(event.target.value)}
6 />
```

`answerText` is the current value. `setAnswerText` changes it and tells React to render again. The input is called *controlled* because React state is its source of truth.

3.4.3 Effects

An effect runs code because a component appeared or a dependency changed. It is the correct place to load API data or install a timer:

```
1  useEffect(() => {  
2    const refresh = () => apiRequest('/leaderboard')  
3    void refresh()  
4    const interval = window.setInterval(refresh, 15000)  
5    return () => window.clearInterval(interval)  
6  }, [selectedSeasonId])
```

The dependency array means the effect restarts when `selectedSeasonId` changes. The returned cleanup function prevents an abandoned timer from continuing after the component disappears.

React development mode uses `StrictMode`, which may run effect setup/cleanup twice to expose unsafe effects. This is intentional and does not happen the same way in a production build.

Frontend architecture

4.1 Starting React

The browser initially receives `index.html`, which contains an empty root element. `src/main.tsx` attaches React to it:

`src/main.tsx`

```
1 const container = document.getElementById('root')!
2 createRoot(container).render(
3   <StrictMode>
4     <App />
5   </StrictMode>,
6 )
```

The exclamation mark tells TypeScript that the element definitely exists. If `index.html` stopped containing `id="root"`, that assertion would be wrong and startup would fail.

4.2 Routing and global login state

`App.tsx` owns the logged-in user because many pages need it. It initializes from browser `localStorage`, then keeps storage synchronized:

`src/App.tsx`, shortened

```
1 const [user, setUser] = useState<UserState>(() => {
2   const storedUser = localStorage.getItem('potw-user')
3   if (!storedUser) return null
4   const parsed = JSON.parse(storedUser)
5   return parsed?.token ? parsed : null
6 })
7
8 useEffect(() => {
9   if (user) localStorage.setItem('potw-user', JSON.stringify(user))
10  else localStorage.removeItem('potw-user')
11 }, [user])
```

React Router selects a page without reloading the whole site:

```

1 <Routes>
2   <Route path="/" element={<FrontPage user={user} setUser={setUser} />} />
3   <Route path="/profile" element={<ViewProfile user={user} setUser={setUser} />} />
4   <Route path="/admin" element={<Admin user={user} setUser={setUser} />} />
5 </Routes>

```

This is a single-page application: navigation changes components and the address bar, but usually does not request a new HTML document.

4.3 The shared API helper

Every frontend request should normally go through `src/lib/api.ts`:

`src/lib/api.ts`, shortened

```

1 export async function apiRequest(path, options = {}, user) {
2   const headers = new Headers(options.headers)
3   if (user?.token) {
4     headers.set('Authorization', `Bearer ${user.token}`)
5   }
6   if (options.body && !(options.body instanceof FormData)) {
7     headers.set('Content-Type', 'application/json')
8   }
9
10  const response = await fetch(`/api${path}`, { ...options, headers })
11  const data = await response.json().catch(() => ({}))
12  if (!response.ok) throw new Error(data.message || 'Something went wrong.')
13  return data
14 }

```

Centralizing this prevents every page from reimplementing token headers and error handling. Notice that it does *not* manually set `Content-Type` for `FormData`; the browser must generate the multipart boundary itself.

4.4 Development proxy

Frontend code requests `/api/...` from port 5173. `vite.config.ts` forwards those requests to Express:

`vite.config.ts`

```

1 server: {
2   proxy: {
3     '/api': 'http://localhost:3000',
4   },
5 }

```

Without this proxy, development requests would hit Vite instead of Express. In production, the host should route `/api` to the backend, or the frontend should be built with `VITE_API_URL` pointing at the API origin.

4.5 Pages and reusable components

The navbar and current-problem widget are components because multiple pages or concerns use them. The page components organize full screens. This division is practical rather than magical: split a component when it has a clear job, is reused, or has grown difficult to read.

The current-problem component demonstrates most common React patterns:

- `useState` stores loaded problems, form values, expanded areas, files, and messages.
- `useEffect` loads public problem data once.
- Derived variables choose the active computational or proof problem.
- Event handlers validate and submit forms.
- Conditional JSX shows login prompts, previews, files, or errors.
- MathJax turns LaTeX strings into rendered mathematics.

Backend architecture and middleware

5.1 What Express adds to Node

Node can create an HTTP server directly, but Express supplies routing and middleware conventions. A route says, “when this HTTP method and path arrive, run this function.”

```
1 app.get('/api/seasons', async (_req, res, next) => {  
2   try {  
3     const result = await query(  
4       'SELECT id, name, start_date, end_date, is_active FROM seasons'  
5     )  
6     res.json({ seasons: result.rows })  
7   } catch (error) {  
8     next(error)  
9   }  
10 })
```

req is the incoming request. res constructs the response. next hands control to the next matching middleware, especially for errors.

5.2 Middleware without the jargon

Middleware is a function placed in the request pipeline. It can inspect/change the request, stop it by sending a response, or call next() to continue.

Imagine airport checkpoints. The passenger is the request. One checkpoint checks the ticket, another scans luggage, and the gate agent performs the final action. Any checkpoint can reject the passenger; otherwise it passes the passenger forward. Middleware is simply those checkpoints written as functions.

The order matters. The app installs broad middleware first:

server/index.js, simplified

```
1 app.use(cors({ origin: process.env.FRONTEND_URL })))  
2 app.use(express.json({ limit: '100kb' })))  
3 app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, limit: 300 })))
```



```
4
5 app.post(
6   '/api/submissions',
7   requireAuth,
8   upload.single('file'),
9   async (req, res, next) => { /* final route logic */ },
10  )
```

For the submission route, the order is:

1. global CORS and request-rate rules;
2. `requireAuth`, which checks the token and sets `req.user`;
3. Multer, which parses one uploaded file into `req.file`;
4. the final handler, which validates and inserts the submission.

5.3 Common middleware in this project

cors Permits the configured frontend origin to call the API from a browser. It is a browser access rule, not authentication.

express.json Parses a JSON request body into `req.body`. It also rejects JSON above 100 KB.

express-rate-limit Counts requests per client IP over a window and returns HTTP 429 after the limit.

requireAuth Validates a Bearer JWT, reloads the user from PostgreSQL, requires verified email, and assigns `req.user`.

requireAdmin Requires `req.user.is_admin` or superuser status.

requireSuperuser Requires the single higher-privilege superuser flag.

multer Parses multipart/form-data and keeps one file in memory, with a 5 MB cap.

error middleware The final four-argument function converts known upload errors and unexpected exceptions into JSON responses.

5.4 The final error handler

```
1 app.use((error, _req, res, _next) => {
2   if (error instanceof multer.MulterError) {
3     return res.status(400).json({ message: 'Files must be 5 MB or smaller.' })
4   }
5   console.error(error)
6   res.status(500).json({ message: 'The server could not complete that request.' })
7 })
```

The four parameters tell Express this is error middleware. Detailed errors go to the server console; users receive a generic message so internal details and secrets are not leaked.

PostgreSQL and how data is stored

6.1 Tables, rows, columns, and keys

A PostgreSQL table resembles a spreadsheet with stronger rules. A row is one record; a column is one field. A primary key uniquely identifies a row. A foreign key connects one table to another and prevents references to nonexistent records.

For example, `submissions.user_id` references `users.id`. The submission stores the numeric relationship instead of repeating a username. If the username changes, the relationship remains valid.

6.2 The connection pool

server/db.js

```
1  const { Pool } = pg
2
3  export const pool = new Pool({
4    connectionString: process.env.DATABASE_URL,
5    ssl: process.env.NODE_ENV === 'production'
6      ? { rejectUnauthorized: false }
7      : false,
8  })
9
10 export function query(text, params) {
11   return pool.query(text, params)
12 }
```

A pool reuses several database connections instead of opening a new connection for every request. `query` is a tiny convenience wrapper used throughout the server.

6.3 Why SQL parameters matter

Never insert user text into an SQL string with concatenation. Use placeholders:

```
1  const result = await query(
2    'SELECT id, username FROM users WHERE LOWER(email) = LOWER($1)',
3    [email],
4  )
```

PostgreSQL receives the command and value separately. It treats `email` as data, even if it contains quote marks or SQL-looking text. This is the primary defense against SQL injection.

6.4 Data dictionary

6.4.1 users

One row per account.

Column	Type	Meaning
<code>id</code>	integer	Generated primary key.
<code>username, email</code>	text	Unique login identity; email is normalized to lowercase on registration.
<code>password_hash</code>	text	Bcrypt hash, never the original password.
<code>is_admin</code>	boolean	May grade and manage problems, proposals, hints, and seasons.
<code>is_superuser</code>	boolean	Includes admin ability and may grant/revoke admin status.
<code>email_verified</code>	boolean	Blocks login until true for newly registered accounts.
<code>email_verification_*</code>	text/timestamps	SHA-256 token hash, expiry, and most recent send time.
<code>password_reset_*</code>	text/timestamp	One-time reset token hash and expiry.
<code>leaderboard_hidden</code>	boolean	Excludes this user from public rankings without deleting scores.
<code>created_at</code>	timestamp	Account creation time.

6.4.2 problems

One row per published, scheduled, draft, or archived problem.

Column	Type	Meaning
<code>id, title</code>	integer/text	Identity and visible title.
<code>statement_latex</code>	text	Public problem body rendered by MathJax.
<code>solution_latex</code>	text	Private reference solution returned only through admin routes.
<code>problem_source, proposed_by</code>	text	Optional credit information.
<code>problem_type</code>	text	Computational or proof-based; controls response UI and validation.
<code>is_current</code>	boolean	Candidate for the current-problem endpoint.
<code>is_archived</code>	boolean	Explicitly placed into the archive.
<code>release_date</code>	date	Calendar date used for seasons and ordering.

Column	Type	Meaning
release_at	timestampz	Exact public release instant. Admin input is interpreted in Indianapolis time.
due_date	date	Calendar date retained for compatibility and display fallback.
due_at	timestampz	Exact instant when submissions and edits close.
hints	text	Optional published LaTeX hint retained for future use.
hints_enabled	boolean	Stored published-hint setting; the current front page does not show hints.
allow_hint_requests	boolean	Whether solvers can send private requests.
difficulty_rating	integer	Whole number from 1 through 10.

6.4.3 submissions

One row per submitted attempt. Multiple attempts are allowed.

Column	Type	Meaning
user_id, problem_id	integer	Foreign keys connecting the solver and problem.
answer_text	text	Computational short answer.
work_text	text	Optional shown work or typed proof, with LaTeX support in the UI.
file_name, file_type	text	Original name and MIME type for an upload.
file_data	bytea	Raw uploaded file bytes stored inside PostgreSQL.
score	integer	0–5 points, with a database constraint.
is_correct	boolean	True only for a score of 5 in the grading route.
feedback	text	Admin feedback visible in the solver's profile.
graded_by, graded_at	integer/timestamp	Admin relationship and grading time; null means pending.
submitted_at	timestamp	Creation time, updated when the solver edits a response.

6.4.4 seasons

Defines a leaderboard date range: name, start date, end date, and active flag. A constraint requires the end not to precede the start.

6.4.5 problem_proposals

Stores solver-suggested title, statement LaTeX, optional solution/source/notes, status, creator, and time. Status is constrained to pending, accepted, or declined.

6.4.6 hint_requests

Connects a user and problem with an optional message, time, and status. Status is constrained to pending or resolved.

6.5 Joins

A submission does not contain its problem title or username. A JOIN combines related rows for display:

```
1 SELECT s.id, s.answer_text, s.score, s.feedback,  
2        p.title, p.problem_type  
3 FROM submissions s  
4 JOIN problems p ON p.id = s.problem_id  
5 WHERE s.user_id = $1  
6 ORDER BY s.submitted_at DESC;
```

Aliases *s* and *p* shorten table names. The *ON* condition explains how rows correspond.

6.6 Schema versus migrations

`001_schema.sql` describes a fresh database. Later numbered files modify databases created before new features existed. Apply them in numeric order. Migrations use patterns such as `ADD COLUMN IF NOT EXISTS` so an already-upgraded local database does not fail immediately.

Do not casually edit an old migration after production has used it. Add a new numbered migration so every environment can move from the same old state to the same new state.

Authentication, verification, and permissions

7.1 Password storage

The backend hashes passwords with bcrypt at cost 12:

```
1 const passwordHash = await bcrypt.hash(password, 12)
```

Login uses `bcrypt.compare`. Hashing is intentionally expensive, which slows password guessing. A hash is not encryption: there is no decryption key and the original password should never be recoverable.

7.2 Email verification flow

1. Registration validates username, email, and password.
2. The server creates 32 cryptographically random bytes and represents them as 64 hexadecimal characters.
3. It stores only `SHA-256(token)`, with a 24-hour expiry.
4. Nodemailer sends the raw token inside a frontend link.
5. The verification page posts the token back to the API.
6. The server hashes the supplied token and matches the stored hash and expiry.
7. A match sets `email_verified = TRUE` and deletes verification token fields.

Storing only the hash means a database reader cannot directly use an unexpired token. In development without SMTP, the server prints and returns a test URL. In production, missing SMTP is an error and test URLs are not exposed.

7.3 Password reset flow

Forgot-password always returns the same public message whether an email exists. This reduces account enumeration. For an existing account it creates a one-hour hashed token and emails the raw link. A valid reset changes the bcrypt hash, verifies the email (because possession of the inbox was demonstrated), and clears the token.

7.4 JWT login sessions

After a verified login, `server/auth.js` signs a JSON Web Token:

`server/auth.js`, shortened

```
1 export function createToken(user) {  
2   return jwt.sign(  
3     { sub: user.id, username: user.username },  
4     process.env.JWT_SECRET,  
5     { expiresIn: '7d' },  
6   )  
7 }
```

The frontend stores the token and sends it as `Authorization: Bearer <token>`. A JWT is signed, not secret: do not put passwords or sensitive records inside its payload.

7.5 Why authentication reloads the user

`server/auth.js`, simplified

```
1 const payload = jwt.verify(token, getSecret())  
2 const result = await query(  
3   'SELECT id, is_admin, is_superuser, email_verified FROM users WHERE id = $1',  
4   [Number(payload.sub)],  
5 )  
6 req.user = result.rows[0]  
7 next()
```

Every protected request reloads roles from PostgreSQL. If the superuser removes admin access, an old seven-day token does not retain admin permission. It also ensures deleted or newly unverified accounts cannot continue merely because an old token exists.

7.6 Authentication versus authorization

Authentication answers “Who are you?” Authorization answers “Are you allowed to do this?”

- Public routes require neither.
- Solver routes use `requireAuth`.
- Grading/problem-management routes use `requireAuth`, `requireAdmin`.
- Role-management routes use `requireAuth`, `requireSuperuser`.

Hiding an admin tab in React is not security. A visitor can manufacture HTTP requests. The backend middleware is the real enforcement boundary.

7.7 Rate limiting and DDoS reality

The API has a broad limit plus tighter limits for registration, failed login, and email actions. This reduces simple automated abuse and prevents one client from generating unlimited email.

In-process rate limiting is not complete DDoS protection. A serious production deployment should put the app behind a CDN/reverse proxy such as Cloudflare, configure trusted proxy handling correctly, restrict body sizes, monitor traffic, and use a shared rate-limit store if multiple API instances run. Email verification discourages fake accounts; it does not stop raw traffic from reaching the server.

Feature workflows

8.1 Problem scheduling

The admin form defaults new problems to the next Monday at 6:00 PM Eastern and the following Sunday at 6:00 PM Eastern. It sends timezone-free local values such as 2026-09-07T18:00. PostgreSQL interprets each value explicitly in `America/Indiana/Indianapolis` and stores absolute `timestampz` instants.

The public current-problem query requires:

- `is_current = TRUE;`
- `is_archived = FALSE;`
- release time has arrived; and
- exact due time has not passed.

The separate `release_date` remains useful for season date ranges and human display. Existing problems were migrated to midnight on their old release date; the weekly due-time migration sets legacy weekly deadlines to 6:00 PM Eastern.

8.2 Computational and proof submissions

Frontend validation improves usability, but backend validation is authoritative:

- computational problems require `answer_text`;
- proof problems require typed `work_text` or a file;
- an optional comment/work field and file may support partial credit;
- files are limited to one and 5 MB;
- the problem must currently be released, current, unarchived, and not overdue.

Answer and work previews are rendered by MathJax in the browser. PostgreSQL stores the original LaTeX text, not rendered HTML or an image. Rendering at display time keeps the data editable.

8.3 Editing a response

The owner may edit text while the problem is still current and before its due date. Saving an edit clears correctness, score, feedback, grader, and grading time, then returns the attempt to the pending queue. An existing uploaded file stays attached.

This reset matters: otherwise a solver could change a correct answer after receiving five points while retaining the grade.

8.4 Grading

Admins load pending submissions, inspect text/files and the private reference solution, then assign an integer score from 0 through 5 and optional feedback. The route sets `is_correct` true only at 5 points and records the grader and time.

The profile refreshes every 15 seconds. It shows:

- exactly what the solver typed and the attached filename;
- Pending, Incorrect, Partial credit, or Correct status;
- score out of 5 and admin feedback;
- a download action for the solver's own file.

8.5 Leaderboard seasons

The chosen season supplies a start and end date. The backend selects problems whose `release_date` falls inside that range. For every user/problem pair it takes `MAX(score)` among graded attempts, so a worse repeat attempt does not erase a better score.

It builds a score map keyed conceptually as `userId:problemId`, then returns one row per visible solver with each problem's score, full-credit count, and total points. Rows sort by total points, then solved count, then username.

Admins and superusers are excluded. Users with `leaderboard_hidden = TRUE` are also excluded, but their submissions and private profile scores remain untouched.

8.6 Hints and proposals

Hint requests live on their own `/hints` page rather than inside the problem card. Solvers choose a problem with title buttons and describe what they tried with a LaTeX preview. The front page currently hides published hints. A solver cannot create another pending request for the same problem, and admins can mark requests resolved.

Problem proposals preserve the solver's LaTeX source so admins can review and copy it. Proposal status changes do not automatically create a problem; an admin deliberately transfers/edits accepted content in the Problems tab.

API reference

Method	Path	Access	Purpose
GET	/api/health	Public	Test API and database connectivity.
POST	/api/auth/register	Public, limited	Create unverified account and send email.
POST	/api/auth/verify-email	Public, limited	Consume verification token.
POST	/api/auth/resend-verification	Public, limited	Send a new verification token when eligible.
POST	/api/auth/login	Public, failed attempts limited	Check password/verification and return JWT.
POST	/api/auth/forgot-password	Public, limited	Request generic reset email response.
POST	/api/auth/reset-password	Public, limited	Consume token and replace password.
GET	/api/problems/current	Public	Released, active, non-overdue problems.
GET	/api/problems/archive	Public	Past/non-current/archived problems.
POST	/api/submissions	Verified user	Submit text and optional file.
GET	/api/submissions/mine	Verified user	Private history with grades.
PATCH	/api/submissions/:id	Owner	Edit eligible response and reset grade.
GET	/api/submissions/:id/file	Owner/admin	Download stored file.
PATCH	/api/profile/preferences	Verified user	Toggle leaderboard privacy.
GET	/api/seasons	Public	List leaderboard seasons.
GET	/api/leaderboard?season_id=:id	Public	Spreadsheet data for one season.
POST	/api/problem-proposals	Verified user	Suggest a problem.
GET	/api/problem-proposals/mine	Verified user	View own proposal statuses.
POST	/api/problems/:id/hint-requests	Verified user	Ask for a hint when enabled.
GET	/api/admin/submissions	Admin	Load pending or all attempts.
PATCH	/api/admin/submissions/:id/grade	Admin	Save score and feedback.
GET/POST	/api/admin/problems	Admin	List/create problems.
PUT/DELETE	/api/admin/problems/:id	Admin	Edit/remove a problem.

Method	Path	Access	Purpose
GET	/api/admin/proposals	Admin	Load proposals and hint requests.
PATCH	/api/admin/proposals/:id	Admin	Change proposal status.
PATCH	/api/admin/hint-requests/:id	Admin	Resolve/reopen hint request.
POST/PUT	/api/admin/seasons[:id]	Admin	Create/update seasons.
GET/PATCH	/api/admin/users[:id]	Superuser	List users or toggle admin role.

Configuration, running, and testing

10.1 Environment variables

Variable	Meaning
DATABASE_URL	PostgreSQL connection URL, including database user, password, host, port, and database name.
JWT_SECRET	Long random signing secret for login tokens. Changing it logs everyone out.
PORT	API port, default 3000.
FRONTEND_URL	Allowed browser origin and base for emailed links.
SMTP_HOST, SMTP_PORT	Mail provider server and port.
SMTP_SECURE	True for immediate TLS, normally port 465; port 587 generally upgrades with STARTTLS.
SMTP_USER, SMTP_PASS	Mail provider credentials.
EMAIL_FROM	Visible sender identity.
NODE_ENV	Set to production in deployment; changes database SSL and email fallback behavior.
TRUST_PROXY	Configure only to match the production reverse-proxy topology so client IP limiting is correct.

10.2 Development commands

From the potw directory:

```
npm install
npm run dev:all
```

This runs `node --watch server/index.js` and Vite together. Keep the terminal open. Visit <http://localhost:5173>. Stop both with Ctrl+C.

Useful checks:

```
npm run lint      # catches suspicious JS/TS/React patterns
```

```
npm run build      # type-checks and produces dist/  
npm audit          # scans installed dependency versions
```

10.3 Database setup

For a new database, apply numbered SQL files in order. The README has copyable commands. If potw already exists, do not recreate it; apply only missing migrations. Back up important data before schema changes.

10.4 A practical manual test checklist

1. Open `/api/health`; expect `{"status":"ok"}`.
2. Register with a temporary email. Confirm login is blocked before verification.
3. Open the development verification link or delivered email and log in.
4. Submit a computational answer with LaTeX preview and optional typed/uploaded work.
5. Submit a proof by text or file; verify an empty proof is rejected.
6. As admin, grade 0, partial, and 5-point attempts and supply feedback.
7. As solver, verify profile statuses, exact response, download, score, and feedback.
8. Toggle Anonymous mode and verify the leaderboard excludes/includes the solver while profile data stays.
9. Create a problem scheduled a few minutes ahead; verify it is absent before and present after the time.
10. Request a password reset; confirm the old password fails and new one works.
11. Run lint, build, and dependency audit before deployment.

How to trace and change the code

11.1 Trace a feature vertically

Do not search randomly. Follow the layers. For “anonymous leaderboard mode”:

1. UI: checkbox in `ViewProfile.tsx`.
2. Frontend request: `PATCH /profile/preferences` through `api.ts`.
3. Backend route: `PATCH /api/profile/preferences` in `server/index.js`.
4. Data: `users.leaderboard_hidden` in `001_schema.sql/migration 005`.
5. Consumer: leaderboard user query filters `leaderboard_hidden = FALSE`.
6. Type: `leaderboard_hidden` in `src/types/user.ts`.
7. Style: `profile-preferences` rules in `Page.css`.

Most full-stack changes touch several of these layers. If the database stores a field but the API never returns it, React cannot display it. If React sends a field but the backend ignores it, it will not persist.

11.2 Example: adding a new problem field

Suppose you add an optional topic such as “Geometry.” A safe sequence is:

1. Add migration 007 with `ALTER TABLE problems ADD COLUMN topic text`.
2. Add it to the fresh schema so new installations also have it.
3. Include it in admin `SELECT/INSERT/UPDATE` code.
4. Validate it on the backend if restrictions exist.
5. Include it in public problem columns if solvers may see it.
6. Add it to the `TypeScript Problem` type.
7. Add an admin form control and display location.

8. Apply the migration, then lint/build/test create/edit/public behavior.

11.3 Reading an error

Work from the first meaningful error, not the last cascade.

Browser console error Usually frontend JavaScript, rendering, or request failure. Check the Network tab response body.

HTTP 400 The request reached the backend but failed validation.

HTTP 401 Token missing, invalid, or expired.

HTTP 403 Identity is known but lacks verification/permission.

HTTP 404 Route or requested record was not found.

HTTP 409 Conflict such as duplicate username/email or pending hint request.

HTTP 429 Rate limit reached.

HTTP 500 Unexpected backend/database/email failure. Read the server terminal stack trace.

ECONNREFUSED The target process is not listening, often API or PostgreSQL not running.

Known tradeoffs and production notes

12.1 File storage

Uploads are currently PostgreSQL bytea. This is simple and keeps ownership/deletion transactional, but large volume makes database backups and memory usage heavy. Multer holds the entire file in server memory. For a larger deployment, store objects in S3-compatible storage, keep only object meta-data/key in PostgreSQL, scan uploads, and use signed download URLs.

12.2 Token storage in localStorage

The JWT lives in `localStorage`, which is straightforward but readable by JavaScript on the page. A cross-site scripting bug could steal it. A hardened design often uses secure, HTTP-only, same-site cookies plus CSRF protections and a strict Content Security Policy.

12.3 One large backend file

`server/index.js` is understandable at this size but is growing. A future refactor could split routes into `routes/auth.js`, `routes/problems.js`, and so on, and move business rules into services. Refactor only with tests; moving code without behavioral tests can silently change middleware order.

12.4 Validation

Validation is currently handwritten. A schema library could centralize body shapes and error messages. Database constraints remain valuable even with application validation because they protect data from bugs and manual SQL.

12.5 Time handling

Release and due instants are timezone-aware and interpreted in Indianapolis time. Calendar-date columns remain for season grouping and compatibility. Time behavior should be tested around daylight-saving changes.

12.6 Operational protection

A production launch should add:

- HTTPS everywhere;
- managed PostgreSQL backups and restore drills;
- a reverse proxy/CDN with request and bot controls;
- a shared rate-limit store for multiple API instances;
- structured logs, uptime checks, and error monitoring;
- verified SMTP sender domain with SPF, DKIM, and DMARC;
- secret rotation and least-privilege database credentials;
- automated API/integration tests and a deployment pipeline;
- dependency update and vulnerability review.

Glossary

API A defined way for programs to communicate. Here it is the collection of `/api/...` HTTP routes.

Bearer token A credential sent in an HTTP header; possession grants the represented session access.

Component A React function that returns part of the interface.

CORS Browser policy controlling which origins may call an API. Not a login system.

CRUD Create, read, update, delete—the basic persistent-data operations.

Environment variable Configuration supplied outside source code, accessed through `process.env` on Node.

Express Node web framework used for routes and middleware.

Foreign key Database constraint connecting a value to a primary key in another table.

HTTP Request/response protocol used between browser and backend.

JSON Text data format used by most API requests/responses.

JWT Signed token containing claims such as user ID and expiration.

JSX HTML-like syntax inside React JavaScript/TypeScript.

LaTeX/MathJax LaTeX text is stored; MathJax renders mathematics in the browser. This guide itself is typeset with LaTeX.

Middleware A function in an Express request pipeline that checks, changes, rejects, or forwards a request.

Migration Ordered SQL change that upgrades an existing database schema.

Node.js Runtime that executes backend JavaScript.

ORM A library mapping database rows to language objects. This app does not use one; it uses SQL through `pg`.

PostgreSQL Relational database system used for durable application data.

Promise JavaScript object representing a future success or failure.

Props Inputs passed from a parent React component to a child.

React Frontend library that renders components from props and state.

Route Combination of HTTP method and URL path connected to code.

SQL injection Attack caused by mixing untrusted text into SQL syntax; parameterized queries prevent it.

State Values React remembers for a component and uses to render.

SMTP Standard protocol used to send account emails.

TypeScript JavaScript with compile-time type checking.

Vite Development server and frontend build tool.

Fast file lookup

Question	Start here
Where is a browser URL mapped?	<code>src/App.tsx</code>
How is the logged-in user remembered?	<code>src/App.tsx</code> , <code>src/types/user.ts</code>
How are API requests made?	<code>src/lib/api.ts</code> , then the calling page
How are current problems/submission UI rendered?	<code>src/components/problem-widget/CurrentProb.tsx</code>
How does grading/problem scheduling work?	<code>src/pages/Admin.tsx</code> and admin routes in <code>server/index.js</code>
Where is an API URL implemented?	Search <code>server/index.js</code> for the path
How is a token checked?	<code>server/auth.js</code>
How is PostgreSQL reached?	<code>server/db.js</code>
How are emails constructed?	<code>server/email.js</code>
What fields are stored?	<code>db/001_schema.sql</code> and this guide's data dictionary
How did an old database gain a field?	Matching numbered migration in <code>db/</code>
How is appearance controlled?	CSS beside the component/page, plus <code>Page.css</code>
What command runs the project?	<code>package.json</code> scripts and <code>README.md</code>

Final mental model

React does not talk to PostgreSQL. It renders state and sends HTTP requests. Express receives those requests, runs ordered middleware, validates identity and data, executes parameterized SQL through a connection pool, and returns JSON. PostgreSQL preserves normalized records and constraints. Email verification and password reset are one-time-token workflows; JWTs represent logged-in sessions; admin middleware enforces permissions. Once that chain is clear, the codebase stops being a pile of unfamiliar frameworks and becomes the same request/data/render cycle repeated for each feature.